

Weekly Announcements

- **Office Hours Request Link:** <https://forms.gle/cQStrQGdnsVZYPgT7>
 - **MP 1** is posted on Canvas under **Assignments**.
 - MP 1 - Part 1 and Part 2 are available.
 - **MP1-WSU 2** is due on Friday (9/25) and will be available the night before.
-

Class 4: Pathlib Basics and Data Formats

Learning Objectives

By the end of today's class, you will be able to:

1. Use `pathlib` to handle file paths that work across operating systems.
2. Explain different data formats: CSV, JSON, YAML, and `.env`.
3. Write separate functions for reading common data formats.

Note: All exercises we do in class are expected to be completed inside the class-exercise directory. Please make sure to navigate there (e.g., `cd class-exercise`) before starting any coding practice during class.

Part 1: Pathlib: Filepath that works anywhere

The old way (string manipulation):

```
import os

# Messy string concatenation
data_dir = 'data'
filename = 'sales.csv'

# Only works on Mac/Linux!
file_path = data_dir + '/' + filename
# Only works on Windows
file_path = data_dir + '\\ ' + filename
```

Better way: pathlib

```
from pathlib import Path

# Clean object-oriented approach
data_dir = Path('data')
file_path = data_dir / 'sales.csv' # Works everywhere!
```

```
# Check if exists
if file_path.exists():
    print("File exists")

# Get extension
ext = file_path.suffix # '.csv'
```

Quick Note on the Forward Slash /

When you use the forward slash (/) between a `Path` object and a string, Python interprets it as a path-joining operator, not mathematical division.

Pathlib Basics (quiz)

```
import logging
from pathlib import Path

logging.basicConfig(
    level=logging.DEBUG,
    format='%(asctime)s %(levelname)-8s %(message)s',
    datefmt='%H:%M:%S'
)
logger = logging.getLogger(__name__)

# 1. Creating paths
file_path = Path('data') / 'sales.csv'
config_path = Path('config') / 'settings.yaml'

# 2. Path properties
print(file_path.name)    # 'sales.csv'
print(file_path.stem)    # 'sales'
print(file_path.suffix)  # '.csv'
print(file_path.parent)  # 'data'

# 3. Checking existence – log the outcome
if not file_path.exists():
    logger.warning(f"File does not exist: {file_path}")
else:
    logger.info(f"File found: {file_path.name}")

# 4. Checking whether a path is specifically a file
if file_path.is_file():
    logger.info(f"{file_path.name} is a file")
```

Part 2: Working with Common File Formats

Different file formats are designed for different types of data.

Format	Common Use	Typical Python Representation
CSV	Tabular data	DataFrame
JSON	Structured data / APIs	Dictionary
YAML	Configuration	Dictionary
<code>.env</code>	Sensitive configuration values	Environment variables

CSV (Quiz)

- CSV/TSV stores data in rows and columns.
- CSV data is naturally tabular, so it can be loaded directly into a pandas DataFrame.

```
#sample.csv
Student_ID,Score
1,10
2,15
3,10
4,10
5,14
```

Read CSV

```
import pandas as pd
df = pd.read_csv("sample.csv")
# TSV
# df = pd.read_csv('data.tsv', sep='\t')
print(df)
```

Write CSV

```
df.to_csv("output.csv", index=False)
```

JSON

- JSON stores structured data using dictionaries and lists. For example,

```
# sample.json
{"Status": "success",
 "Count": 3,
 "Data": [{"name": "Alice", "city": "Boston"},
          {"name": "Bob", "city": "New York"}]}
```

```
    {"name": "Charlie", "city": "Chicago"}]
}
```

Read JSON

```
import json

with open("sample.json", "r") as f:
    data = json.load(f)

print(data["Status"])
print(data["data"])
```

Write JSON

```
with open("output.json", "w") as f:
    json.dump(data, f, indent=2)
```

YAML

YAML is commonly used for configuration files. For example:

```
# sample.yaml
cleaning:
  missing: "drop"

processing:
  batch_size: 100
```

Read YAML

```
import yaml

with open("sample.yaml", "r") as f:
    config = yaml.safe_load(f)

print(config["cleaning"]["missing"])
print(config["processing"]["batch_size"])
```

Configuration data is usually kept as a dictionary.

.env: Sensitive Credentials in Configuration

What is **.env**? A **.env** file (dot-environment) is a simple plain-text file used to store sensitive configuration and environment variable.

Important: Do not commit **.env** to GitHub.

```
# .env
USERNAME=ds3500demo
PASSWORD=ds3500
API_KEY=12345abcde
```

Load the environment variables in Python:

```
import os
from dotenv import load_dotenv

load_dotenv()

username = os.getenv("USERNAME")
password = os.getenv("PASSWORD")
api_key = os.getenv("API_KEY")

print(password)
# Never print passwords, API keys, or other secrets in a real application.
```

Summary:

Data	File	Example
Non-sensitive settings	.yaml	Processing, cleaning
Sensitive information	.env	Passwords, API keys, tokens
Code logic	.py files	Functions, classes

Try It Yourself — Inspect Different Data Formats

After completing this section

- Raise your hand and check in with either me or a TA.
- Once we have confirmed that you have completed the section, you may leave.

Before You Start

Open your **class-exercise** folder

Inside the folder:

- Create a new Python file named `class4_format_inspector.py`.
- Create a `.env` file (in the same root folder as `class4_format_inspector.py`) with a couple of sample values:

```
USERNAME=ds3500demo
PASSWORD=ds3500
API_KEY=12345abcde
```

- Create a `data/` folder and sample files inside the folder:

```
data/
├── sample.csv
├── sample.json
└── sample.yaml
```

- `sample.csv`

```
Student_ID,Score
1,10
2,15
3,10
4,10
5,14
```

- `sample.json`

```
{"Status": "success",
 "Count": 3,
 "Data": [{"name": "Alice", "city": "Boston"},
           {"name": "Bob", "city": "New York"},
           {"name": "Charlie", "city": "Chicago"}]
}
```

- `sample.yaml`

```
cleaning:
  missing: "drop"

processing:
  batch_size: 100
```

Reminder: `.env` should never be committed to GitHub — make sure it's listed in your `.gitignore` before you commit anything.

Task

Write four separate inspection functions, `inspect_csv(filepath)`, `inspect_json(filepath)`, `inspect_yaml(filepath)`, and `inspect_env()`. Each data inspection function should log which filepath it is inspecting. Use `print()` for the file contents because the contents are program output, not log messages.

In `main()`, use `pathlib` to create the data file paths. Create a `Path` for the `data/` directory, then use the `/` operator to build the CSV, JSON, and YAML paths. Keep the `.env` loading separate and simple using `load_dotenv()`.

Complete the TODOs.

```
import json
import logging
from pathlib import Path

import pandas as pd
import yaml
import os
from dotenv import load_dotenv

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s %(levelname)-8s %(message)s",
    datefmt="%H:%M:%S"
)
logger = logging.getLogger(__name__)

def inspect_csv(filepath):
    """Read a CSV file and display basic information."""
    # TODO:
    # 1. Read the file using pd.read_csv().
    # 2. Log the filepath at INFO.
    # 3. Print the first three rows (e.g. DataFrame.head(3))
    pass

def inspect_json(filepath):
    """Read a JSON file and display basic information."""
    # TODO:
    # 1. Open the file and read it using json.load().
    # 2. Log the filepath at INFO.
    # 3. Print the contents.
    pass

def inspect_yaml(filepath):
```

```
"""Read a YAML file and display basic information."""
# TODO:
# 1. Open the file and read it using yaml.safe_load().
# 2. Log the filepath at INFO.
# 3. Print the contents.
pass

def inspect_env():
    """Read a .env file and display basic information."""
    load_dotenv()

    keys = [
        key for key in ["USERNAME", "PASSWORD"]
        if os.getenv(key) is not None
    ]

    # TODO:
    # 1. Log at INFO that .env was loaded.
    # 2. Print keys.
    # Do not print passwords, API keys, or other secret values.

def main():
    # TODO:
    # 1. Create a Path object for the data directory.
    # 2. Use the / operator to build the CSV, JSON, and YAML paths.
    # 3. Call each inspection function using the matching path.
    # 4. Call inspect_env() without an argument.
    pass

if __name__ == "__main__":
    main()
```

Try These Commands

```
python class4_format_inspector.py
```

Expected output:

```
10:15:02 INFO    Inspecting CSV: data/products.csv
...
10:15:02 INFO    Inspecting JSON: data/products.json
...
10:15:02 INFO    Inspecting YAML: data/sample.yaml
...
10:15:02 INFO    Loaded environment variables from .env
...
```

Once everything looks good, please raise your hand and check in with us!

Save Your Work to GitHub

Before committing, double check `.env` is being ignored:

```
git status
# .env should NOT appear in the list of files to be committed
```

When you finish:

```
git add class4_format_inspector.py
git commit -m "Compare common data formats"
git push
```
